MOESI-D: A Persistent Sole-Writer Cache-Coherence State for Commutative No-Return Atomics

Manav Sharma
Independent Researcher

Yaseen Alkhameri
Independent Researcher

***Abstract*—Parallel workloads in machine learning like sparse embedding updates and gradient aggregation, graph analytics, and streaming telemetry rely heavily on atomic read–modify–writes to shared counters. Although each update modifies only a few bytes, conventional cache-coherence protocols repeatedly bounce entire cache lines between processor cores, stalling execution pipelines and saturating on-chip interconnects.**

We introduce MOESI-D, an architectural extension that adds a persistent Delegated (D) state to the MOESI protocol, moving the update to the data rather than migrating the line to each core. Targeting commutative, no-return integer reductions (e.g., addition, min/max, bitwise masks), MOESI-D designates a single cache controller as the sole writer for a contended line. Remote updates are forwarded as compact messages and applied in place via a lightweight controller-side ALU. Because issuing cores do not require a return value, operations complete fire-and-forget, allowing out-of-order pipelines to proceed immediately while store buffers preserve memory consistency.

We implemented MOESI-D in gem5 and evaluated it against conventional atomics, software flat combining, and privatization techniques across synthetic microbenchmarks, sparse kernels, and telemetry workloads. MOESI-D eliminates 99.9% of cache-line migrations under heavy contention. On out-of-order cores, fire-and-forget execution yields an approximate $2\times$ speedup over standard atomics, scaling to $5.3\times$ when the hot working set exceeds L2 cache capacity. Detailed network simulations further demonstrate an approximate $2.1\times$ reduction in on-chip traffic, demonstrating that hardware delegation drastically lowers communication and execution costs for shared accumulation.

***Index Terms*—Cache coherence, atomic memory operations, hardware delegation, gem5, multi-core architectures, sparse computing.**

I. Introduction

Cache coherence is optimized for read-sharing and single-writer locality. It is ill-suited for a narrower but pervasive access pattern: **many cores updating the same line with small atomic read–modify–writes**. Examples include histogram and Count-Min-Sketch accumulation, sparse-matrix–vector (SpMV) and graph-analytics reductions, high-rate abuse counters, and telemetry aggregation. In the prior gem5 study by Le et al. [17], SpMV and GAPBS BFS workloads show that a two-state MI protocol drives exclusive-ownership traffic and directory forwards nearly 1:1 into tens of millions of transactions (e.g., HB/685_bus at 4 cores: 12.69M `GETX`, 12.27M `FWD_GETX`; HB/bcsstm12 up to 156M `FWD_GETX` at 8 cores). MESI cuts `GETX` via Exclusive and Shared states by 313–1685$\times$, and MOESI's Owned state further reduces read misses and invalidations. That study also reports a **read-forwarding traffic spike** (`FWD_GETS` increases $\sim$20$\times$ on HB/685_bus), which motivated its conceptual Delegated-state proposal [17]. These read-forwarding measurements are distinct from the write-induced line migrations evaluated in the present work.

Two specific structural properties make line migration unnecessary for these updates:

1) **Commutativity and Associativity:** For operations such as integer `add`, `min`, `max`, `and`, `or`, and `xor`, applying an update wherever the line currently resides produces identical architectural state compared to migrating the line first.
2) **No-Return Semantics:** The issuing thread does not consume the previous value (e.g., x86 `LOCK ADD` vs. `XADD`, RISC-V `AMO... rd=x0`, ARMv8.1-LSE `STADD`). Because no instruction depends on an immediate return value, the update does not need to complete synchronously.

Building on the conceptual Delegated (D) state proposed by Le et al. [17], MOESI-D exploits both properties. Rather than transferring cache-line ownership on each write, the directory promotes a contended line to a **Delegated (D)** state. One node is designated as the **sole writer**; peer cores forward their updates as small, low-overhead messages ($\sim$32 B) that a dedicated mini-ALU at the sole writer's cache controller executes in place. Cache-line bouncing is eliminated. Furthermore, because the issuing core requires no return value, the instruction completes **fire-and-forget**: the pipeline is released immediately at issue time, while memory consistency is preserved via a store-buffer slot. On an out-of-order core, this mechanism converts reduced network traffic directly into execution latency reduction.

Importantly, D-state acts as a strict performance layer: any delegated operation can be NACKed and subsequently re-issued via the conventional `GETX`-based atomic path, guaranteeing forward progress and correctness.

No institutional resources were used in this work. Implementation and evaluation on gem5 v25.1.0.1. Artifact available at: https://github.com/msharma2026/MOESI-D-Architecture.

Contributions:

- A directory-gated **Delegated coherence state** for commutative no-return AMOs featuring a sole-writer mini-ALU, fire-and-forget pipeline retirement, and a fail-safe fallback path, fully implemented in gem5 Ruby/SLICC with an x86 `stula` micro-op.
- An implemented extension of the conceptual Delegated state, motivated by the prior MI, MESI, and MOESI characterization on SpMV and GAPBS BFS [17], with evaluation of contended no-return atomic updates.
- Quantitative comparisons against naive atomics, software flat combining, and privatization, including the structural verification that D-state remains inert on cleanly row-partitioned SpMV.
- An out-of-order microarchitectural evaluation over large key spaces showing an $\approx 2\times$ latency advantage on OoO cores, scaling up to $\approx \mathbf{5.3}\times$ under cache capacity overflow, and an $\approx \mathbf{2.1}\times$ reduction in on-chip network flit injection.
- Protocol extensions including an **adaptive keep-D** policy for read-interleaved workloads and capacity-eviction mechanisms robust up to $4\times$ the L2 cache size.

II. Background and Motivation

A. MOESI and the Line-Migration Overhead

In standard directory-based MOESI, each cache line resides in Modified (M), Owned (O), Exclusive (E), Shared (S), or Invalid (I) state. The directory serializes requests, issuing `FWD_GETS` and `FWD_GETX` messages to current owners. When multiple cores repeatedly issue atomic updates to distinct words within the same cache line, each store triggers a `FWD_GETX`, forcing a complete 128-byte cache-line transfer. On out-of-order processors, returning atomics (`XADD`, `CMPXCHG`) serialize execution, stalling pipeline retirement until the entire line migrates and provides the old value.

B. Commutative, No-Return Atomics

Commutative no-return AMOs represent a distinct workload class standardized across multiple ISAs and language specifications (e.g., C++26 P3111 atomic reduction operations). Because execution order and execution site do not alter the final result, and because the calling thread discards the pre-update value, the microarchitecture is freed from synchronous line ownership transfers.

C. Design Space of Prior Approaches

Table I contextualizes MOESI-D within prior architectural techniques for contended atomic updates.

MOESI-D introduces a persistent, directory-arbitrated coherence state specifically targeting commutative no-return operations via controller-side execution, functioning conceptually as hardware-enforced flat combining without software locking overhead.

TABLE I
Design Space of Hardware Support for Contended Updates

Strategy	Representative	Execution Site
Privatization	COUP [1], COMMTM [2]	Local core copy
In-Situ Atomics	Atomic Cache [15]	8T SRAM (GPU)
Asynch. Reduct.	Tascade [9]	NoC Routers
Coherence Queue	QOLB [16]	L1/L2 linked lists
Mem-side PIM	AMD US 11,726,918 [6]	DRAM PIM
Per-op AMO	DynAMO [4], Delegato [5]	Near-core vs. LLC
Remote Atomics	Intel RAO [7]	Memory controller
Softw. Combine	Flat Combining [8]	Single thread lock
Delegation	**MOESI-D (This work)**	**Cache controller**

III. The Delegated (D) State

A. Core Mechanism

We implement the sole-writer delegation concept outlined by Le et al. [17] using the following protocol mechanism. When contention is detected on a line, the directory designates the current owner as the **sole writer** and transitions the line to the D state. Subsequent commutative AMOs from remote cores are forwarded as compact `FWD_DSTATE_REQ` messages ($\sim$32 bytes containing the opcode, operand, word offset, requester ID, and transaction tag). As illustrated in Fig. 1, the sole writer applies the mutation locally using an integrated mini-ALU and replies with an acknowledgment; the 128-byte line remains stationary. Requesters holding the line locally in state D execute the update via a zero-network fast path.

B. Cache-Controller Mini-ALU

Every cache controller incorporates a fixed-function arithmetic logic unit capable of executing 32-bit/64-bit integer additions, bitwise operations, and min/max comparisons. The mini-ALU operates on the cache controller's internal event queue, fully decoupled from the main processor pipeline. This decoupling guarantees deadlock freedom during mutual cross-delegation, ensuring incoming forwarded requests are drained even if the local CPU core is stalled. Transitions adhere to an *apply-then-ACK* discipline, maintaining serializability.

C. Promotion Gate and Fallback Mechanics

Promotion to D is selectively gated: the home directory maintains a per-line access counter (`DStateReqCount`) and a bitmask of requesting nodes (`DStateRequestorSet`). Promotion occurs only when the request count reaches a threshold (default 4) and involves ≥ 2 distinct requesters, avoiding unnecessary state transitions for private access streams. If a delegated request encounters a full queue or pending transient transition, a NACK is issued, prompting the requester to safely fall back to a baseline `GETX` acquisition.

D. Fire-and-Forget Completion

Because commutative no-return updates do not deliver results to architectural registers, the issuing core retires the instruction

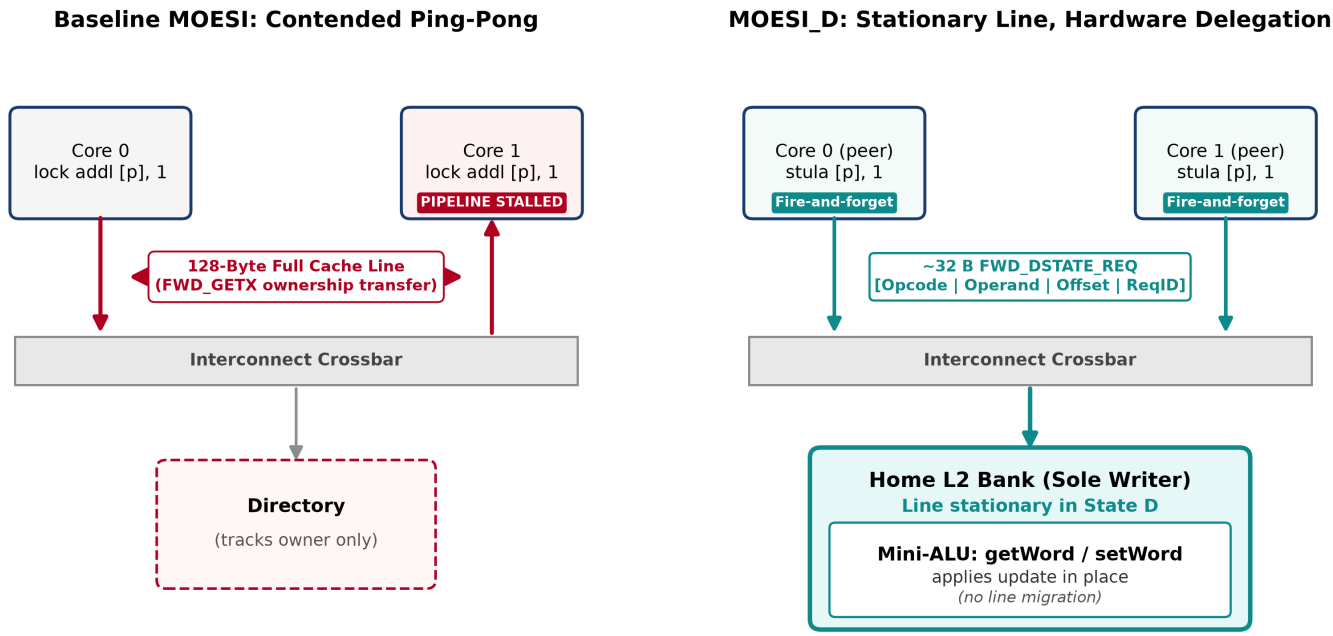


Fig. 1. Conventional Line Migration vs. MOESI-D In-Place Delegation. The baseline MOESI protocol (left) forces 128-byte line migrations on every write, stalling the pipeline. MOESI-D (right) leaves the line stationary in the Delegated (D) state, processing ∼32 B forwarded operations via a cache-controller mini-ALU while the issuing core retires the operation fire-and-forget.

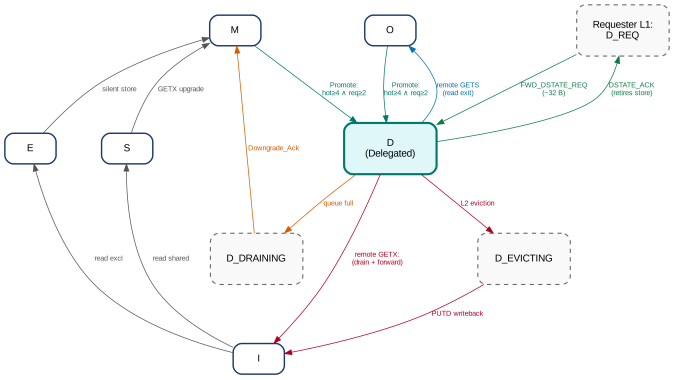


Fig. 2. The Delegated (D) Coherence State Machine. Contended lines promote from Modified or Owned; exits handle reads, forced writes, and evictions cleanly.

immediately upon dispatching the network message. The transient `D_REQ` protocol state monitors completion and absorbs the eventual ACK/NACK. Memory consistency is preserved by allocating a store-buffer entry (`pendingDStateReqs`) that enforces TSO and RVWMO ordering against younger stores and memory fences.

Speculative Safety: Delegated requests are dispatched to the interconnect exclusively at instruction commit, draining from the store buffer after ROB retirement. Consequently, speculative instructions squashed by branch mispredictions or exceptions never inject uncommitted mutations into remote caches.

E. State Transitions and Exits

The D state transitions dynamically based on subsequent traffic, as illustrated in Fig. 2:

- **Read (`GETS`):** Serviced as owner forwarding; the holder demotes to Owned (O) or remains sole-writer under adaptive policy.
- **Exclusive Write (`GETX`):** Forces the D-state holder to drain pending queue operations, invalidate its copy, and forward the 128-byte line (D → I).
- **Congestion / Eviction:** Input queue saturation triggers self-demotion to Modified (M). Evictions follow standard dirty-line writebacks, preserving accumulated state.

F. Deadlock Avoidance

The interconnect is partitioned into three virtual networks: requests (vnet0), forwards (vnet1), and responses (vnet2). The dependency chain remains strictly acyclic (vnet0 → vnet1 → vnet2). Congestion notifications are assigned to vnet2, and `GETS` operations on D lines are handled via owner-forwarding rather than re-entering vnet0.

IV. Implementation

MOESI-D is constructed by extending gem5's Ruby `MOESI_CMP_directory` protocol. We introduce stable state `D`, transient states (`D_REQ`, `IGDX`, `IGDXO`, `IGD_P`, `DX_DRAIN`, `D_DRAINING`, `D_EVICTING`), and explicit message types (`DSTATE_REQ`, `DSTATE_ACK`, `DSTATE_NACK`, `FWD_DSTATE_REQ`). In the SLICC specification, gem5's `DataBlock` is augmented with `getWord` and `setWord`

methods, reducing multi-cycle byte-shift operations to 3-instruction native 32-bit arithmetic.

ISA Modification (`stula`): In stock gem5, x86 `lock addl` is decomposed into a locked read-modify-write micro-op sequence (`ldstl + add + stul`), intercepted by the Ruby sequencer via `isLockedRMW()` before reaching the AMO routing logic. We implemented the **`stula`** (Store Unlocked AMO Add) micro-op, which executes an atomic addition while tagging the memory request with `Request::ATOMIC_NO_RETURN_OP`. Macro-op decoders for `ADD_LOCKED` were repointed to `stula`, enabling stock binary code to trigger delegation. Because `stula` is treated as a no-return store, condition flag evaluation (`ZF`, `CF`, etc.) is suppressed, matching the execution model of ARMv8.1 `STADD`. In the evaluated prototype, the sole-writer mini-ALU is co-located with the home L2 cache bank.

Hardware Overhead: The fixed-function mini-ALU supports basic integer arithmetic and bitwise reductions (IADD32, IMIN/MAX, AND/OR/XOR). Structurally, a narrow fixed-function integer execution unit requires minimal logic and adds negligible complexity relative to the storage and tag overhead of a 128 kB L2 cache bank; detailed physical-design synthesis remains outside the scope of this architectural simulation study.

V. Methodology

Prior Baseline Study: The MI/MESI/MOESI characterization summarized in Fig. 3 is drawn from Le et al. [17], whose setup used 32 kB L1 instruction/data caches. The configuration below describes the present MOESI-D experiments.

Simulation Configuration: Simulations are conducted using gem5 v25.1.0.1. We evaluate both in-order (`X86MinorCPU`) and out-of-order (`X86O3CPU`) execution models. Caches are configured with 128-byte lines, 16 kB direct-mapped L1 instruction/data caches, and a 4-bank address-interleaved 512 kB shared L2 (128 kB per bank). Forwarded operations are processed sequentially by the controller-side mini-ALU. Correctness soaks utilized Ruby's random tester (1/3 reads, 1/3 writes, 1/3 no-return atomics) up to 1M operations, and passed Garnet network integration tests on both Crossbar and 5-core/4-L2 `Mesh_XY` topologies without deadlocks or memory ordering violations.

Baselines:

- **Naive:** Contended execution using standard `lock addl` and `lock xaddl` targeting base MOESI locked-RMW semantics.
- **Flat Combining (FC):** Software synchronization via a designated combiner thread utilizing 128-byte padded publication records and TATAS with pause backoff.
- **Privatization:** Per-thread private accumulation arrays combined via reduction at termination.

Benchmarks:

- `scatter`: N threads concurrently updating 32-bit counters within a single shared 128-byte cache line.
- `scatter_rw`: Contended counters interleaved with periodic full-line reads at varying frequencies.
- `scatter_multi`: Multi-line contention where N counters are padded to distinct 128-byte lines to evaluate working-set capacity effects.
- `litmus`: Synthetic contention targeting shifting hot windows under a Zipfian distribution.
- `cms`: 4-row Count-Min Sketch telemetry under heavy-tailed network traffic.
- `spmv`: Compressed Sparse Row SpMV across SuiteSparse matrices (685_bus, bp_400, bcsstm12) with single-writer row partitions.

VI. Evaluation

A. Coherence Traffic and Protocol Behavior

The prior baseline study characterizes the distinct MI, MESI, and MOESI traffic profiles [17]. In contrast, Table II provides an overview of protocol behavior across key benchmarks using the in-order `X86MinorCPU` model.

TABLE II
In-Order Core Results Summary (D-State vs. Baselines)

Workload	Observed Behavior and Performance Metric
`scatter` (4c)	`FWD_GETX` reduced 80,080 → 92 (99.9% ↓); 21% faster
`litmus` (4c)	D-state beats naive by 23%, priv by 12%, FC by 86%
`litmus` (4c→8c)	D-state execution flat (+1.7%); gap vs priv widens to 25%
Priv-friendly `litmus`	D-state matches batch privatization (+1.7–2.5% delta)
SpMV (all matrices)	`DSTATE_HOT = 0` in all configurations (inert mechanism)

In contended `scatter`, line migrations decrease by 99.9%. Figure 3 instead summarizes the earlier MI/MESI/MOESI characterization [17]; its read-forwarding spike is not the `scatter` line-migration measurement. For cleanly row-partitioned SpMV, the directory observer records `DSTATE_HOT = 0` throughout the computation. Because each row update is confined to a single writer thread, the distinct-requester criterion is never satisfied, and the protocol remains dormant without incurring false delegations.

B. Read-Interleaved Workloads and Adaptive Keep-D

Interleaving read operations into atomic update streams forces conventional lines to demote (D → O). To mitigate recurring re-promotions, an adaptive keep-D mechanism monitors local read runs. Table III illustrates line-migration filtering under varying read frequencies.

TABLE III
Line Migrations (`FWD_GETX`) Under Read Interleaving

Read Frequency	Adaptive Keep-D	Pure Keep-D	Baseline
Every 16 updates	**464**	794	143,174
Every 8 updates	**1,194**	1,186	140,724
Every 4 updates	13,754	6,006	124,814

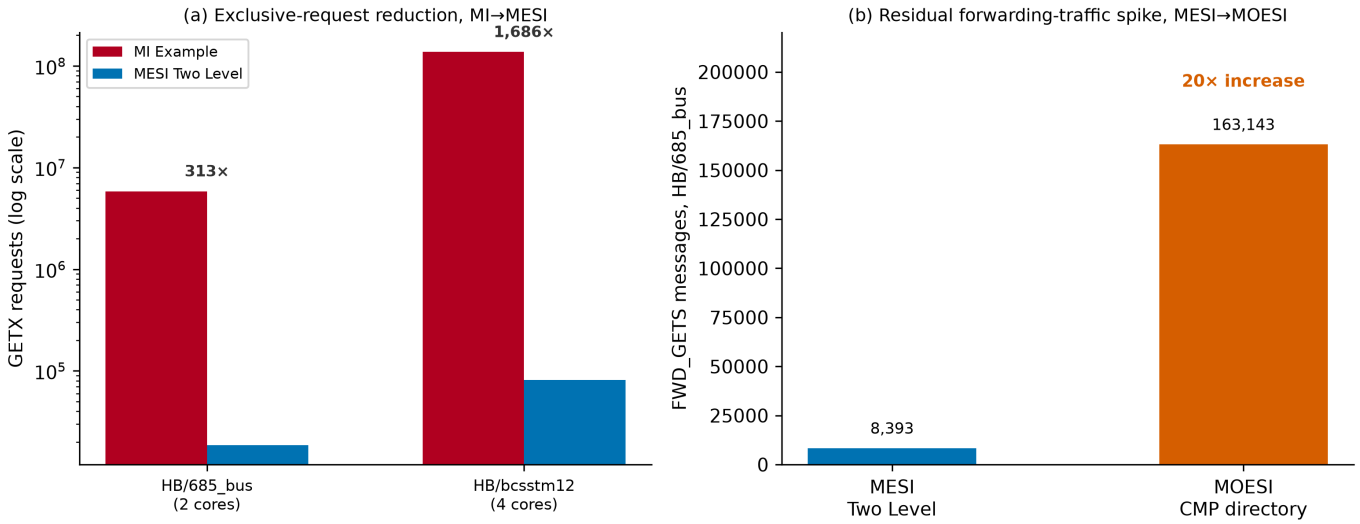


Fig. 3. Baseline Coherence-Traffic Characterization, with data adapted from Le et al. [17]. (a) MESI reduces exclusive ownership requests compared to MI. (b) MOESI increases read-forwarding traffic (`FWD_GETS`) relative to MESI on HB/685_bus. These baseline measurements motivate the present work; they are not MOESI-D results or direct measurements of write-induced line migration.

Adaptive keep-D reduces line migrations by 87–95% during write-dominant phases, reverting to standard shared states when read intensity dominates.

C. Working-Set Capacity Scaling

To assess scaling across memory footprint sizes, `scatter_multi` sweeps the count of active cache lines under out-of-order execution.

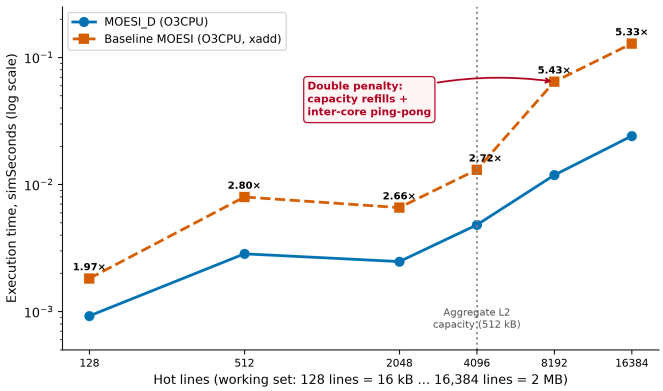


Fig. 4. Working-Set Scaling. When the hot working set exceeds the aggregate L2 capacity (512 kB), the baseline suffers a double penalty of capacity refills and inter-core ping-pong, while D-state's advantage scales to ≈5.3×.

As outlined in Table IV and visualized in Fig. 4, when the footprint fits within the L2 cache, D-state maintains an advantage of ∼2.7×. When the active key space exceeds the aggregate L2 capacity (>512 kB), the baseline encounters compounding performance degradation: lines suffer both DRAM capacity evictions and repeated inter-core ping-pong migrations. In contrast, D-state concentrates operations at the home bank, eliminating line transfers entirely and stabilizing at an ≈ **5.3**× speedup.

TABLE IV
WORKING-SET SCALING ACROSS CACHE BOUNDARIES ON X86O3CPU

Hot Lines	Footprint	D-State (s)	Baseline (s)	Speedup
128	16 kB (L1)	0.000923	0.001819	1.97×
512	64 kB	0.002843	0.007963	2.80×
2,048	256 kB ($\frac{1}{2}$ L2)	0.002470	0.006567	2.66×
4,096	512 kB (L2)	0.004802	0.013054	2.72×
8,192	1 MB (2× L2)	0.011839	0.064302	**5.43**×
16,384	2 MB (4× L2)	0.024084	0.128395	**5.33**×

D. Out-of-Order Latency Amplification

While the in-order CPU exhibits modest runtime improvements (∼1.2×), evaluating MOESI-D on an out-of-order processor (`X86O3CPU`) exposes substantial latency reductions, as detailed in Fig. 5.

TABLE V
SPEEDUP COMPARISON: IN-ORDER VS. OUT-OF-ORDER CORES

Benchmark	In-Order (MinorCPU)	Out-of-Order (O3CPU)
`litmus` (D-state vs. Naive)	1.23×	**2.21**×
`scatter` (D-state vs. `xadd`)	∼1.15×	**2.10–2.75**×

As shown in Table V, out-of-order execution enables fire-and-forget instructions to clear the issue queue immediately, decoupling pipeline throughput from remote coherence stalls and yielding an ≈ **2**× speedup.

E. Multi-Baseline Scaling and Telemetry Workloads

Under heavy contention, software synchronization methods expose severe scalability limits. As shown previously in Fig. 6, MOESI-D effectively flatlines scaling degradation by

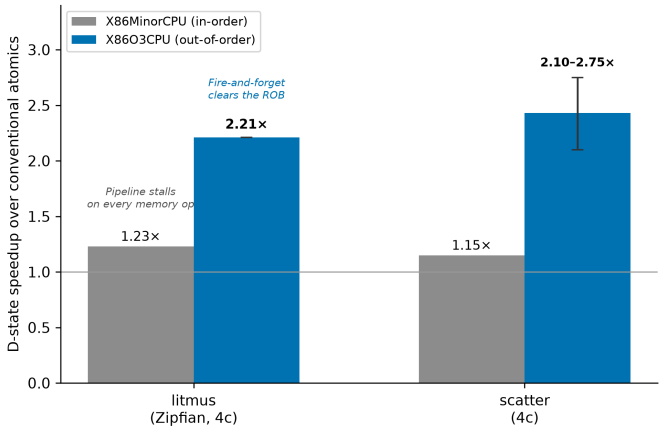


Fig. 5. Out-of-Order Latency Amplification. Fire-and-forget retirement allows out-of-order cores to hide memory latencies, amplifying D-state's performance advantage from $\sim$1.2$\times$ (in-order) to 2.10–2.75$\times$ (out-of-order).

eliminating nearly 99.9% of 128-byte line migrations, while alternatives such as flat combining and privatization suffer lock serialization and merge overheads.

To validate on a real-world workload, we evaluated a Count-Min Sketch (CMS) algorithm used for line-rate DDoS telemetry. Eight cores stream packets, hashing keys into a shared 4×4096 sketch array under heavy-tailed network contention. As shown in Fig. 7, the standard software mitigation—maintaining per-core sketches and periodically flushing them to global memory—actually backfires due to the flush overhead, running 35% slower than naive atomics and 58% slower than MOESI-D. MOESI-D is the only approach that outperforms naive atomics, bypassing both software overhead and line-migration latency.

Interconnect traffic was measured using gem5's Garnet detailed network model on a crossbar topology executing 4-core `scatter`. D-state injected **100,093 flits** compared to **212,052 flits** for the baseline, achieving an $\approx \mathbf{2.1}\times$ reduction in network flit volume while preserving equivalent hop counts.

F. Capacity-Pressure Robustness

Expanding the active working set up to $4\times$ L2 capacity required addressing two critical boundary cases: (1) managing dirty line evictions in state D by redirecting them through transaction writeback buffers to DRAM, and (2) ensuring lines with in-flight fire-and-forget operations defer local eviction until protocol acknowledgment. With these mechanisms active, all benchmark executions completed with bit-exact checksum verification.

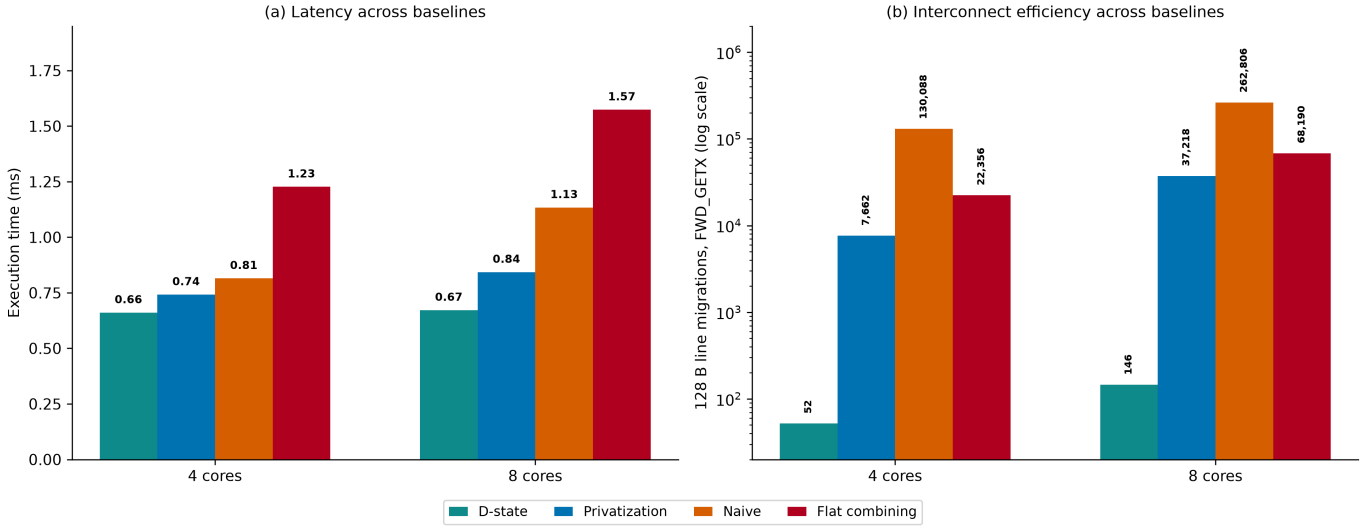


Fig. 6. Unified Multi-Baseline Comparison. D-state eliminates 99.9% of 128-byte line transfers (right), allowing execution time to scale almost perfectly flat from 4 to 8 cores (left), whereas privatization, flat combining, and naive atomics degrade heavily under contention.

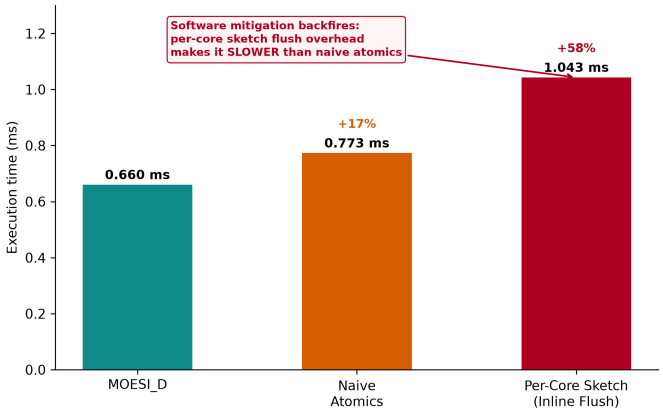


Fig. 7. Count-Min Sketch Telemetry. Standard software mitigations (per-core sketches with periodic flushing) backfire under heavy contention, leaving MOESI-D as the only approach to outperform naive atomics.

VII. Discussion and Limitations

- **Home-Bank Placement:** In this prototype, delegation execution occurs at the home L2 slice rather than at a dynamically assigned processor core. While functionally effective, core-to-core delegation remains an open design point.
- **Workload Scope:** Graph analytics (GAPBS BFS) were characterized under baseline coherence in the prior study [17] to motivate the delegation proposal, but full-scale end-to-end graph algorithm evaluation under MOESI-D remains future work.
- **Consistency Model Interactions:** The latency reduction on out-of-order cores stems in part from fire-and-forget retirement, contrasting with synchronous locked instructions in the baseline.
- **Floating-Point Reductions:** Due to non-associativity in floating-point arithmetic, varying arrival orders may produce minor numerical divergences; exception flags are suppressed during delegation.
- **Power and Energy Accounting:** We report flit count reductions as an interconnect dynamic-energy proxy; comprehensive package-level energy evaluations via McPAT remain future work.

VIII. Related Work

Prior SpMV and Graph Coherence Study: Le et al. [17] compared MI, MESI, and MOESI in gem5 on SpMV and GAPBS BFS and proposed a conceptual Delegated state in which one cache holder acts as the sole writer and other cores send lightweight update instructions. That report did not implement or evaluate the proposed state. The present work builds on that proposal with a gem5 implementation and an evaluation of commutative, no-return atomic updates.

Cache-Level Commutativity: COUP [1] and COMMTM [2] replicate lines across cores, deferring reduction until conflict or read phases. MOESI-D differs by routing operations to a single stationary sole writer, avoiding post-execution reduction passes.

In-Cache and SRAM Atomics: Recent architectures like Atomic Cache [15] modify 8T SRAM arrays to support in-situ arithmetic for GPGPUs, alleviating the von Neumann bottleneck. MOESI-D avoids custom SRAM cell redesigns, relying instead on standard 6T SRAM and a cache-controller mini-ALU, making it suitable for general-purpose CPUs.

Dynamic AMO Routing: DynAMO [4] and Delegato [5] dynamically assign general AMOs between near-core and LLC execution sites. MOESI-D implements a state-machine-driven coherence state explicitly tailored for associative no-return primitives.

Asynchronous Reductions and Queuing: Foundational protocols like QOLB [16] integrated synchronization primitives directly into cache coherence linked lists to eliminate lock spinning. More recently, Tascade [9] constructs asynchronous reduction trees using specialized P-caches for sparse algorithms. In contrast to tree-based network reductions, MOESI-D leverages a persistent coherence state to flatten execution into a single stationary L2 home bank.

Near-Memory Computing: AMD US Patent 11,726,918 [6] couples coalescing caches with DRAM-side PIM. In contrast, MOESI-D manages delegation entirely within the coherent on-chip cache subsystem.

Software and Network Combining: Software flat combining [8] serializes requests via software publication lists, whereas MOESI-D implements line-level combining in hardware, removing locking penalties.

IX. Conclusion

MOESI-D demonstrates that multi-core contention on commutative, no-return atomics can be resolved without line-migration thrashing. By promoting contended lines to a Delegated state with sole-writer mini-ALU execution, the protocol eliminates 99.9% of line migrations. Fire-and-forget retirement provides an $\approx 2\times$ latency advantage on out-of-order cores, expanding to $\approx \mathbf{5.3}\times$ when working sets exceed cache capacity. Furthermore, on-chip flit injection is reduced by $\approx 2.1\times$. The architecture delivers scalable hardware delegation while preserving forward progress, memory consistency, and software transparency.

References

[1] G. Zhang, W. Horn, and D. Sanchez, "Exploiting commutativity to reduce the cost of updates to shared data in cache-coherent systems (COUP)," in *Proc. 48th Int. Symp. Microarchitecture (MICRO)*, 2015, pp. 13–25.

[2] G. Zhang and D. Sanchez, "Leveraging hardware caches for memoization to exploit semantic commutativity in hardware speculation (COMMTM)," in *Proc. 49th Int. Symp. Microarchitecture (MICRO)*, 2016, pp. 1–13.

[3] W. H. Horn, "Cache-coherent commutative operations," M.S. thesis, Massachusetts Institute of Technology, Cambridge, MA, 2015.

[4] R. Soria-Pardos, A. Armejach, D. Suárez-Gracia, et al., "DynAMO: Improving parallelism through dynamic placement of atomic memory operations," in *Proc. 50th Annu. Int. Symp. Computer Architecture (ISCA)*, 2023, pp. 1–14.

[5] R. Soria-Pardos et al., "Delegato: Locality-aware atomic memory operations on chiplets," in *Proc. 58th Int. Symp. Microarchitecture (MICRO)*, 2025.

[6] Advanced Micro Devices, Inc., "Dynamically coalescing atomic memory operations for memory-local computing," U.S. Patent 11,726,918 B2, Aug. 15, 2023.

[7] Intel Corporation, "Remote atomic operations in multi-socket systems," U.S. Patent 10,296,459, May 21, 2019.

[8] D. Hendler, I. Incze, N. Shavit, and M. Tzafrir, "Flat combining and the synchronization-parallelism tradeoff," in *Proc. 22nd ACM Symp. Parallelism in Algorithms and Architectures (SPAA)*, 2010, pp. 355–364.

[9] M. Sutherland et al., "Tascade: Hardware support for atomic-free, asynchronous and efficient reduction trees," *arXiv:2311.15810*, 2023.

[10] A. Gottlieb et al., "The NYU Ultracomputer—Designing an MIMD shared memory parallel computer," *IEEE Trans. Comput.*, vol. C-32, no. 2, pp. 175–189, Feb. 1983.

[11] ARM Limited, "ARM Architecture Reference Manual ARMv8, for ARMv8-A architecture profile," document DDI 0487B.a, 2017.

[12] RISC-V Foundation, "The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2," 2017.

[13] ISO/IEC JTC1/SC22/WG21, "P3111R0: Atomic Reduction Operations," 2024.

[14] J. Lowe-Power et al., "The gem5 Simulator: Version 20.0+," *arXiv:2007.03152*, 2020.

[15] Y. Zhang et al., "Atomic Cache: Enabling Efficient Fine-Grained Synchronization with Relaxed Memory Consistency on GPGPUs Through In-Cache Atomic Operations," in *Proc. 57th Int. Symp. Microarchitecture (MICRO)*, 2024.

[16] J. R. Goodman, M. K. Vernon, and P. J. Woest, “Efficient synchronization primitives for large-scale cache-coherent multiprocessors,” in *Proc. 3rd Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 1989, pp. 64–75.

[17] K. Le, K. Chang, V. L. Nguyen, and Y. Alkhameri, “Cache coherence protocols on sparse-matrix vector multiplication (SpMV) and graphs in GEM5,” unpublished manuscript, University of California, Davis, 2026. [Online]. Available: https://alkhameri.com/papers/cache-coherence-spmv-graphs-gem5.pdf